AIM: A system has RAM which can store four pages simultaneously to satisfy page requests. Write a Program to calculate percentage page hit and page fault if the system uses Optimal page replacement technique for the page references entered by the user i.e. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (No of Frames=3).

INPUTE:

```
GNU nano 8.7                                    optimal.c
#include <stdio.h>

int findOptimal(int pages[], int frames[], int n, int index, int frameCount) {
    int farthest = index, pos = -1;
    for (int i = 0; i < frameCount; i++) {
        int j;
        for (j = index; j < n; j++) {
            if (frames[i] == pages[j]) {
                if (j > farthest) {
                    farthest = j;
                    pos = i;
                }
                break;
            }
        }
        if (j == n) {
            return i; // Page not used again, replace this
        }
    }
    return (pos == -1) ? 0 : pos;
}

int main() {
    int pages[] = {1,2,3,4,1,2,5,1,2,3,4,5};
    int n = sizeof(pages)/sizeof(pages[0]);
    int frameCount = 3;
    int frames[frameCount];
    int count = 0, hits = 0, faults = 0;

    for (int i = 0; i < frameCount; i++) frames[i] = -1;

    for (int i = 0; i < n; i++) {
        int flag = 0;
        for (int j = 0; j < frameCount; j++) {
            if (frames[j] == pages[i]) {
                hits++;
```

```
GNU nano 8.7                                    optimal.c
    int n = sizeof(pages)/sizeof(pages[0]);
    int frameCount = 3;
    int frames[frameCount];
    int count = 0, hits = 0, faults = 0;

    for (int i = 0; i < frameCount; i++) frames[i] = -1;

    for (int i = 0; i < n; i++) {
        int flag = 0;
        for (int j = 0; j < frameCount; j++) {
            if (frames[j] == pages[i]) {
                hits++;
                flag = 1;
                break;
            }
        }
        if (!flag) {
            if (count < frameCount) {
                frames[count++] = pages[i];
            } else {
                int pos = findOptimal(pages, frames, n, i+1, frameCount);
                frames[pos] = pages[i];
            }
            faults++;
        }
    }

    printf("Total Requests: %d\n", n);
    printf("Page Hits: %d\n", hits);
    printf("Page Faults: %d\n", faults);
    printf("Hit Percentage: %.2f%%\n", (hits * 100.0) / n);
    printf("Fault Percentage: %.2f%%\n", (faults * 100.0) / n);

    return 0;
}
```

OUTPUT:

```
KHUSHABU@DESKTOP-VE8LIPH MSYS ~
$ cd

KHUSHABU@DESKTOP-VE8LIPH MSYS ~
$ nano leastrecently.c

KHUSHABU@DESKTOP-VE8LIPH MSYS ~
$ gcc leastrecently.c -o leastrecently

KHUSHABU@DESKTOP-VE8LIPH MSYS ~
$ ./leastrecently
Page Hits = 2
Page Faults = 10
Hit Percentage = 16.67
Fault Percentage = 83.33

KHUSHABU@DESKTOP-VE8LIPH MSYS ~
$
```